



BLM4021 Gömülü Sistemler

Doç. Dr. Ali Can KARACA

ackaraca@yildiz.edu.tr

Yıldız Teknik Üniversitesi – Bilgisayar Mühendisliği

Sunum 7 – Haberleşme Protokolleri

- UART
- I2C
- SPI
- CAN
- Raspberry Pi örnekleri

Gerekli Kaynaklar:

- Derek Molloy, Exploring Raspberry Pi: Interfacing to the Real World with Embedded Linux, Wiley, 2016.
- M. Wolf, Computers as Components: Principles of Embedded Computing System Design, Elsevier, 2008.

Yardımcı Kaynaklar:

- P. Membrey, D. Hows, Learn Raspberry Pi 2 with Linux and Windows 10, Apress, 2015.
- Ali Saidi, The ARM architecture slide.
- Farid Farahmand, Chapter- 3, Embedded Systems with ARM Cortex-M, 2018.
- O. Urhan, Gömülü Sistem Lisansüstü Ders Notları, 2018.
- P. Dutta, Design of Microprocessor-Based Systems, University of Michigan.

Haftalık Konular



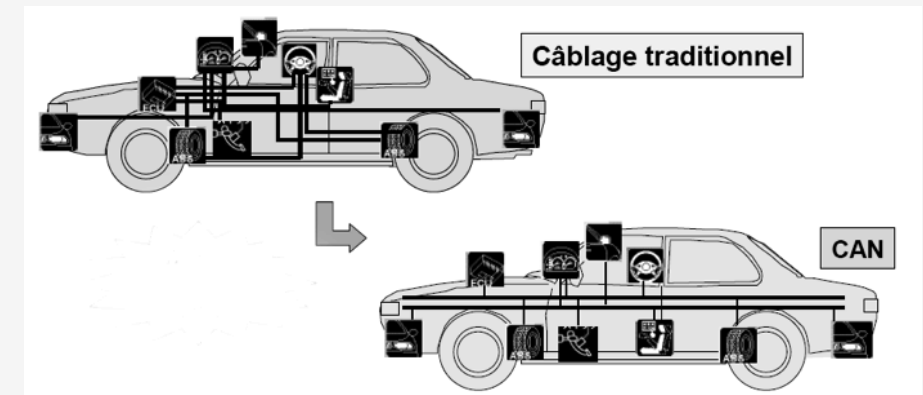
Hafta	Teorik	Laboratuvar
1	Giriş ve Uygulamalar, Mikroişlemci, Mikrodenetleyici ve Gömülü sistem kavramlarının açıklanması	Grupların oluşturulması & Kitlerin Testi
2	Bir Tasarım Örneği, Mikroişlemci, Mikrodenetleyici, DSP, FPGA, ASIC kavramları	Kitlerin gruplara dağıtımı + Raspberry Pi Kurulumu
3	16, 32 ve 64 bitlik mikrodenetleyiciler, pipeline, PIC ve MSP430 özellikleri	Raspberry Pi ile Temel Konfigürasyon
4	ARM tabanlı mikrodenetleyiciler ve özellikleri	---- Resmi Tatil ---
5	ARM Komut setleri ve Assembly Kodları-1	Uygulama 1 – Raspberry Pi ile Buzzer Uygulaması
6	ARM Komut setleri ve Assembly Kodları-2, Raspberry Pi vers. ve GPIO'ları	Uygulama 2 – Raspberry Pi ile Ivme ve Gyro Uygulaması
7	Haberleşme Protokolleri (UART, SPI, I2C ve CAN)	Uygulama 3 – Raspberry Pi ile Motor Kontrol Uygulaması
8	Vize Sınavı	
9	Sensörlerden veri toplama, algılayıcı, ADC ve DAC	Proje Soru-Cevap Saati - 1
10	Zamanlayıcı ve kesmeler	Proje Soru-Cevap Saati -2
11	Görüntü, Ses ve Video	Proje kontrolü-1
12	Gerçek Zaman Sistemlerinde temel kavramlar	Proje Kontrolü-2
13	Gerçek zaman İşletim Sistemleri	Mazeret sebepli son proje kontrollerinin yapılması
14	Nesnelerin İnterneti	
15	Final Sınavı	

For more details -> Bologna page: <http://www.bologna.yildiz.edu.tr/index.php?r=course/view&id=9463&aid=3>

Motivation

- Motivation
 - Connect different systems together
 - Two embedded systems
 - A desktop and an embedded system
 - Connect different chips together in the same embedded system
 - MCU to peripheral
 - MCU to MCU
 - Without using a lot of I/O lines
 - I/O lines require I/O pads which cost \$\$\$ and size
 - I/O lines require PCB area which costs \$\$\$ and size
 - Often at relatively low data rates
 - But sometimes at higher data rates

So, what are our options?



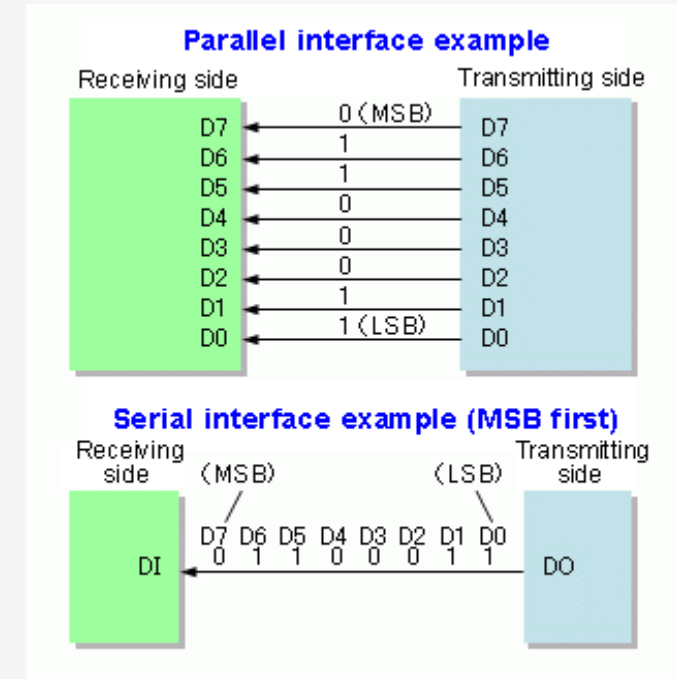
Seri Haberleşme Protokolleri



Paralel haberleşmenin aksine, haberleşme hattından her bir çevrimde bir bit gönderilecek şekilde veri gönderir.

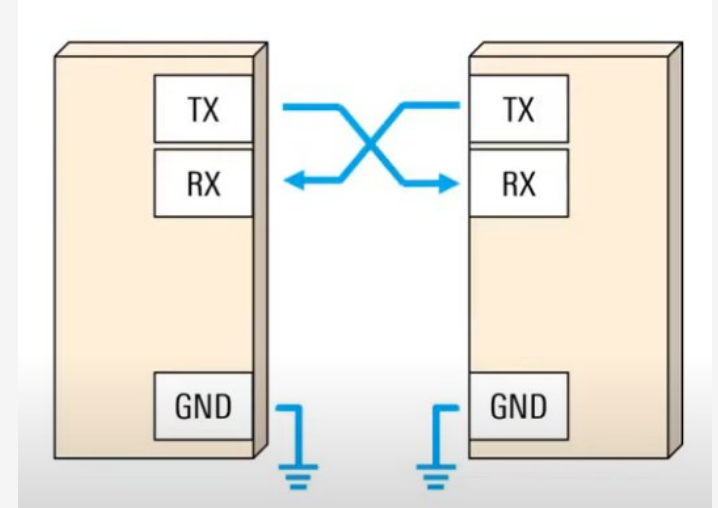
Az sayıda I/O üzerinden gönderilebilir.

- Inter-Integrated Circuit (I2C),
- Serial Peripheral Interface (SPI),
- Controller Area Network (CAN),
- Asenkron seri haberleşme (UART -> RS232, RS485 ve RS422),
- USB

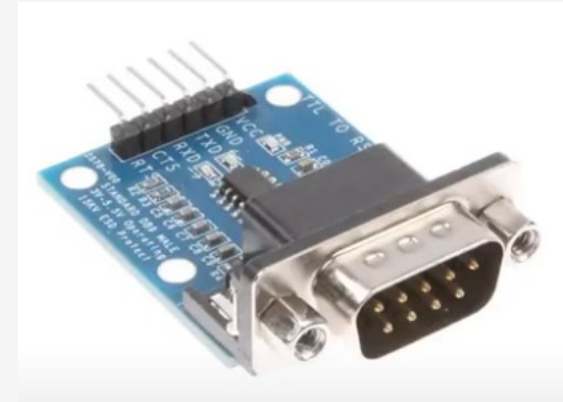


UART (Universal Asynchronous Receiver-Transmitter)

- RX ve TX olmak üzere 2-kablolu hatta ve bir ground hattına ihtiyaç duyar.
- 1'e 1 haberleşme yapılabilir.
- Asenkron haberleşmeye sahiptir.
- 15 metre uzaklığa kadar iletim uzunluğuna sahiptir.
- Haberleşme hızı konfigüre edilebilir.



Protocol	Expandable	Multi-Master	Duplex	Robustness	Speed	Flow Control	Protocol Overhead
UART (2-wire)	No	No	Full	(Parity bit)	~100Kbps	No	Start, Stop, (parity)



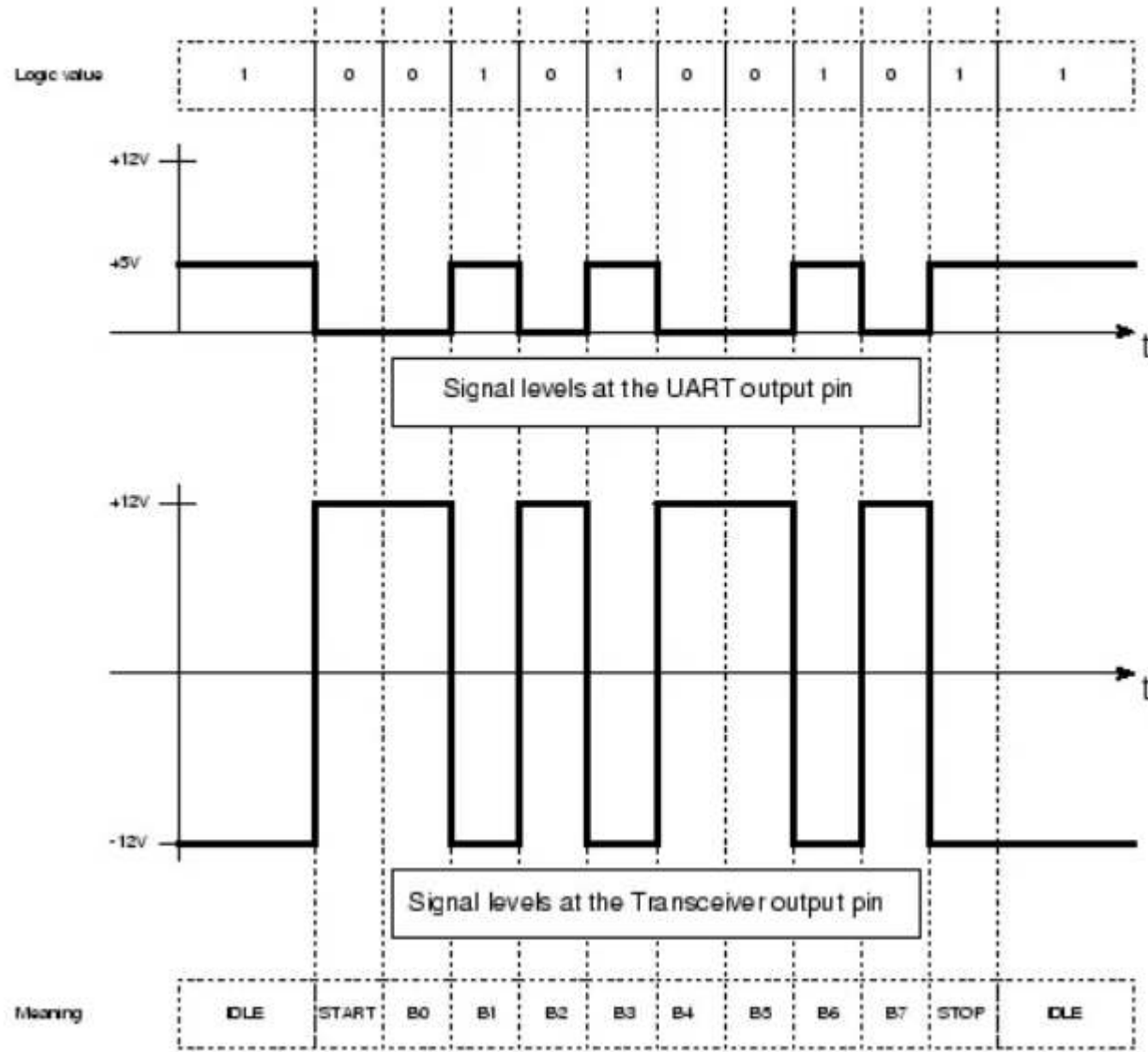
UART nerelerde kullanılıyor?



- En eski seri protokollerden biri olup RS-232, COM portu, modemlerde kullanılmaktadır.
- Elbette gelişen teknoloji ile bilgisayarlarda haberleşme için daha çok Ethernet ve USB, bileşenler arası haberleşme için ise SPI ve I2C kullanılmaktadır.
- Yine de oldukça ucuz bir çözüme sahiptir.
- UART bir protokoldür, gerilim seviyelerin ne olacağı RS-XXX standardı ile belirlenebilir.



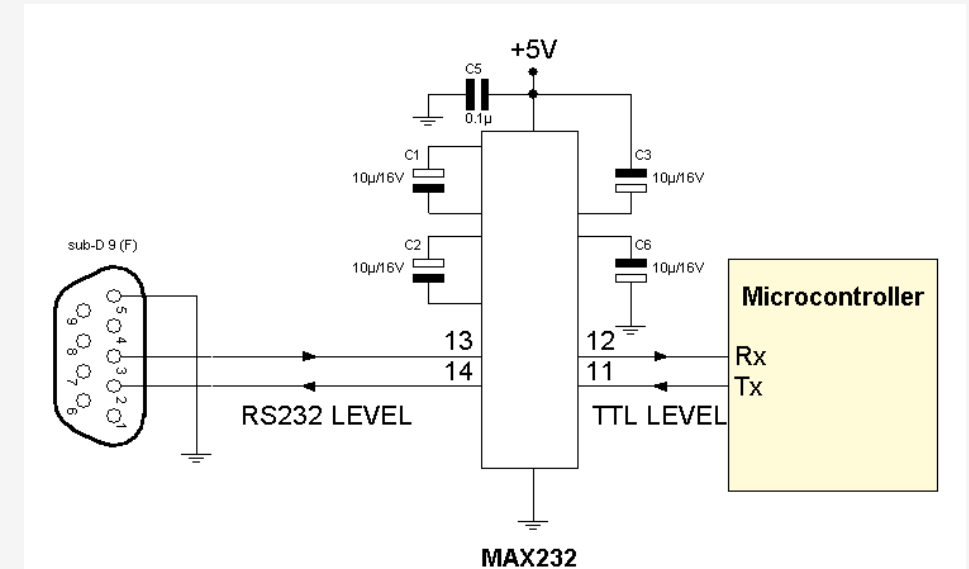
RS232 Transmission of the letter 'J'



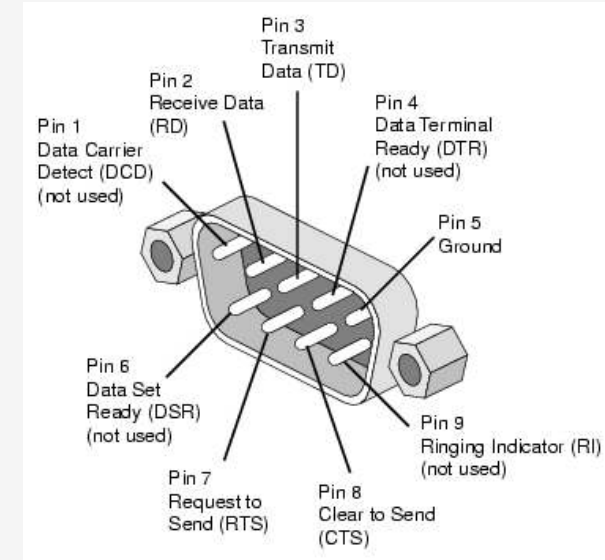
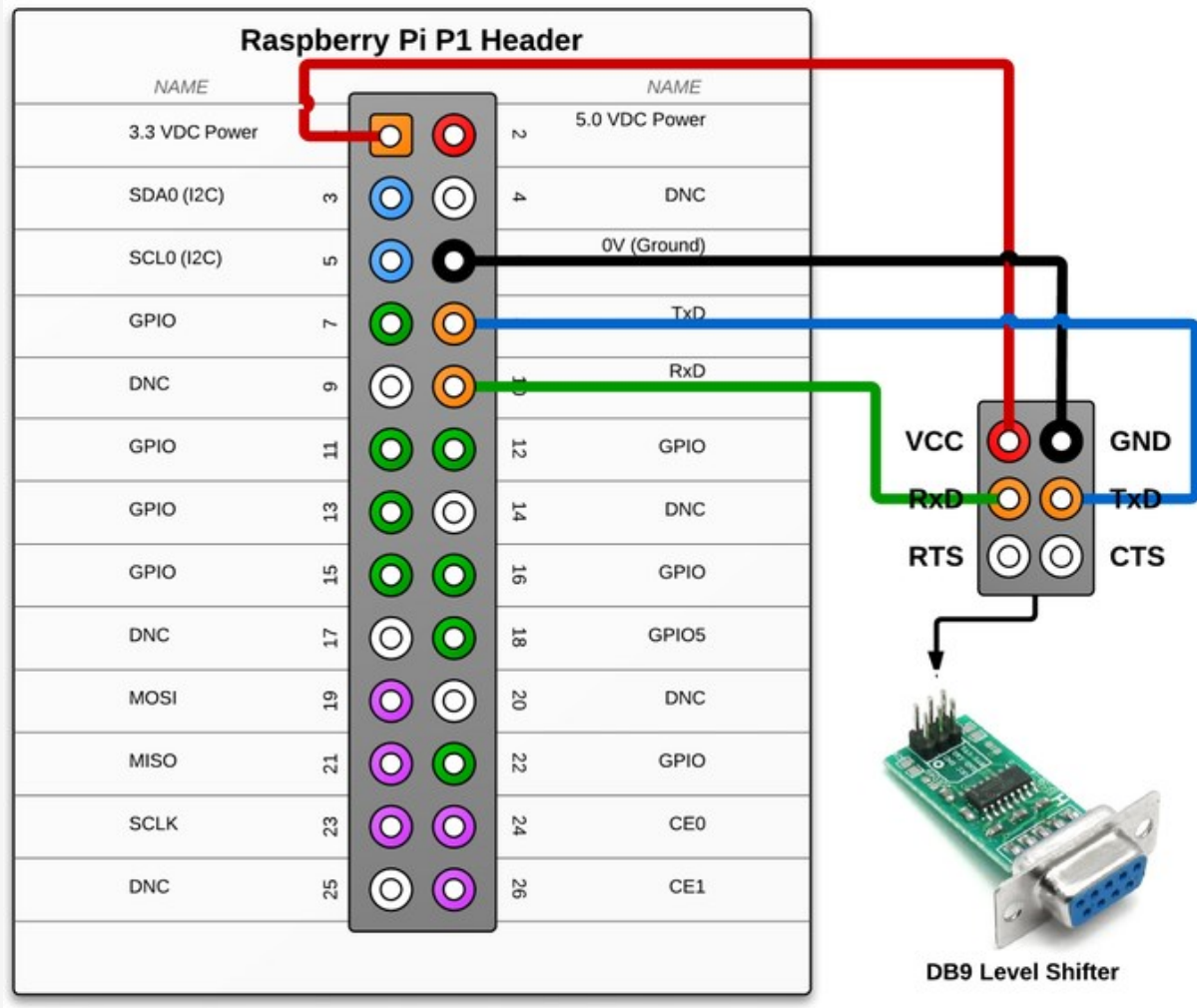
Her ne kadar işlem birimlerinde UART çıkışı olsa da sinyal seviyesinin terslenmesi ve belirli gerilim aralığına getirilmesi gerekmektedir.

Bu çözüm level shifter devreleri ile sağlanmaktadır.

MAX232 bu işi yapan en bilinen entegrelerdendir.



Raspberry Pi - UART



Example: GPS Module using UART

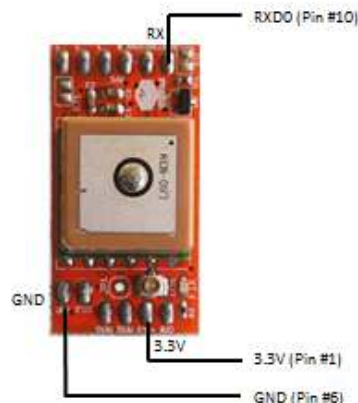
Global **P**ositioning **S**ystem (GPS) makes use of signals sent by satellites in space and ground stations on Earth to accurately determine their position on Earth.

When powered up, they send lines of ASCII text in a special format, called [NMEA](#) messages, that can be received as string with with a simple `readline()` call. Normally the baud rate is 9600 baud by default.

The **Cirotech** module mounted on a Raspberry Pi breakout board with a 40 pin header. It is more sophisticated and contains a real-time clock with battery backup and a 3 axis digital compass using I²C communication. It is available from different distributors (see also on eBay).



Connecting the **Microstack** module is extremely simple, just use 3 wires as shown in the picture here. If you are operating the module outdoor, there is no need for an external antenna. To test the module, you can start a PuTTY terminal (baud rate: 9600 baud) in a VNC session or write a simple Python program that just reads and displays the incoming data. Be aware that for the Raspberry Pi 3 the serial port name is different and you must set the `core_freq` parameter as stated above.



Do not power the module with 5V!



\$GPGGA,1235.19,4807.038,N,01131.000,E,1,08,0.9,545.4,M,46.9,M,,*47

1 2 3 4 5 6 7

1	1235.19	Saat (UTC Biriminde, 13:35:19)
2	4807.038,N	Enlem
3	01131.000,E	Boylam
4	1	0 ise Geçerli Değil, 1 ise GPS, 2 ise DGPS
5	08	Aktif Uydu Bağlantı Sayısı
6	545.4,M	Yükseklik (Metre Biriminde)
7	*47	Checksum

Example: GPS Module using UART

```
import serial                #import serial package
from time import sleep
import webbrowser            #import package for opening link in browser
import sys                  #import system package

def GPS_Info():
    global NMEA_buff
    global lat_in_degrees
    global long_in_degrees
    nmea_time = []
    nmea_latitude = []
    nmea_longitude = []
    nmea_time = NMEA_buff[0]           #extract time from GPGGA string
    nmea_latitude = NMEA_buff[1]        #extract latitude from GPGGA string
    nmea_longitude = NMEA_buff[3]       #extract longitude from GPGGA string

    print("NMEA Time: ", nmea_time, '\n')
    print("NMEA Latitude:", nmea_latitude, "NMEA Longitude:", nmea_longitude, '\n')

    lat = float(nmea_latitude)          #convert string into float for calculation
    longi = float(nmea_longitude)        #convert string into float for calculation

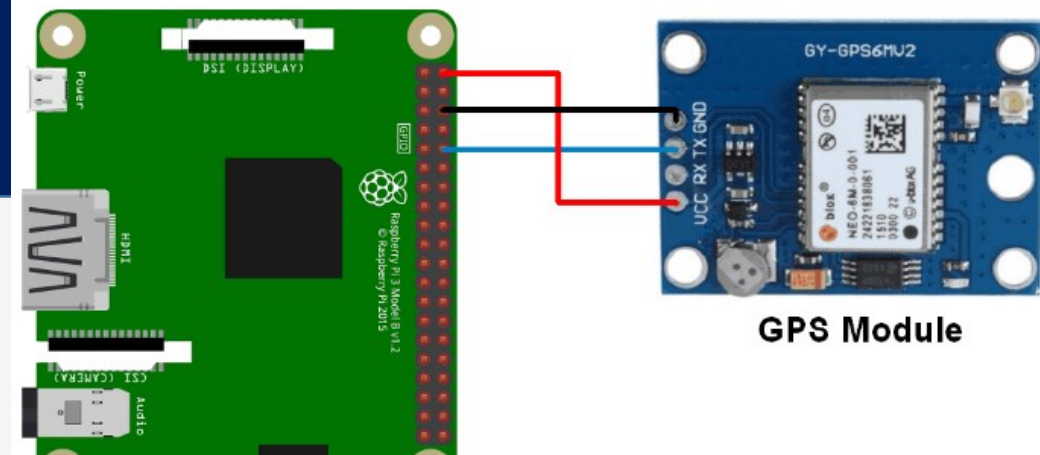
    lat_in_degrees = convert_to_degrees(lat) #get latitude in degree decimal form
    long_in_degrees = convert_to_degrees(longi) #get longitude in degree decimal form
```

```
gpgga_info = "$GPGGA,"
ser = serial.Serial ("/dev/ttyS0")      #Open port with baud rate
GPGGA_buffer = 0
NMEA_buff = 0
lat_in_degrees = 0
long_in_degrees = 0

try:
    while True:
        received_data = (str)(ser.readline()) #read NMEA string
        GPGGA_data_available = received_data.find(gpgga_info) #check for NMEA string
        if (GPGGA_data_available>0):
            GPGGA_buffer = received_data.split("$GPGGA,",1)[1] #store data coming in buffer
            NMEA_buff = (GPGGA_buffer.split(',')) #store comma separated values
            GPS_Info() #get time, latitude, longitude, altitude, quality of fix, etc.

            print("lat in degrees:", lat_in_degrees, " long in degree: ", long_in_degrees)
            map_link = 'http://maps.google.com/?q=' + lat_in_degrees + ',' + long_in_degrees
            print("<<<<<<<<press ctrl+c to plot location on google maps>>>>>>>>\n")
            print("-----\n")

except KeyboardInterrupt:
    webbrowser.open(map_link) #open current position information in google maps
    sys.exit(0)
```



GPS Module

Example: GPS Module using UART



Python 3.4.2 Shell

File Edit Shell Debug Options Windows Help

NMEA Latitude: 1832.9724 NMEA Longitude: 07347.4944

lat in degrees: 18.5495 long in degree: 73.7916

<<<<<<<press ctrl+c to plot location on google maps>>>>>>>

To plot our location on Google map, we need to call URL link for Google map. We can use following link for opening google map with our extracted longitude and latitude.

<http://maps.google.com/?q=<latitude>,<longitude>>

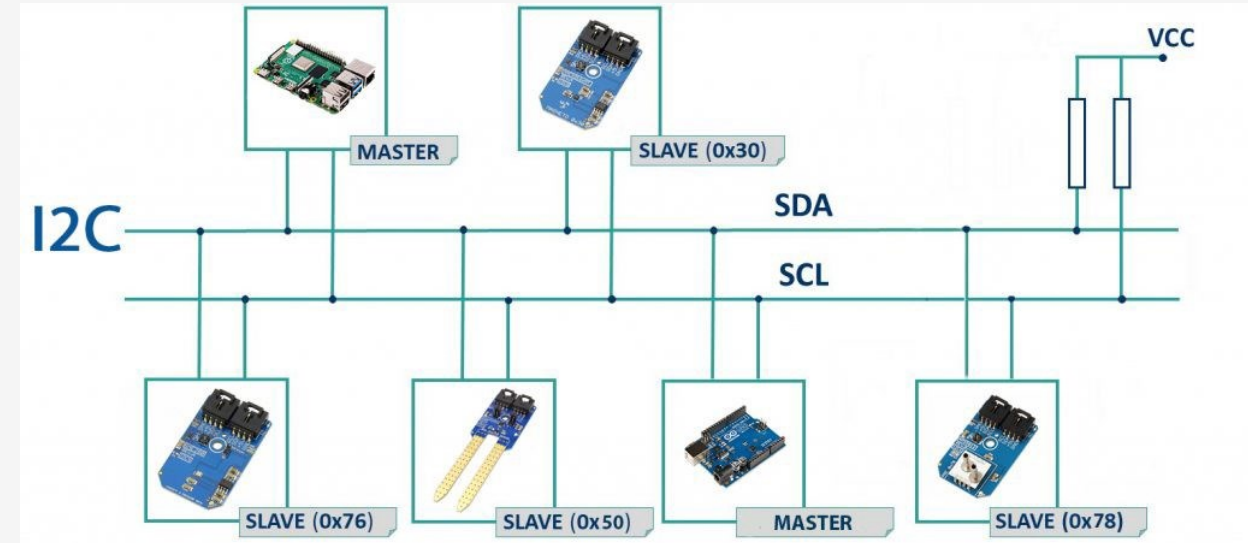
Alternatives:

- You can show the location with markers.
- You should define bus stations and determine the closeness of them.
- ...



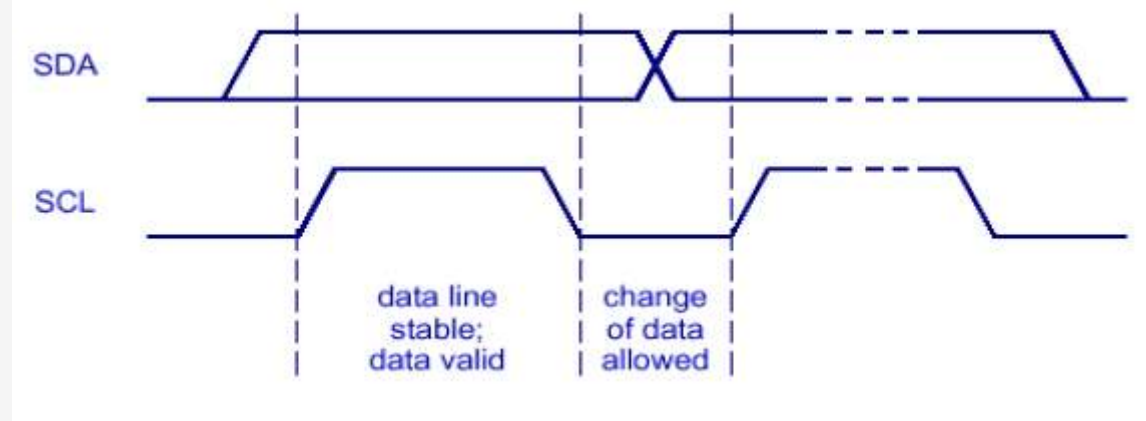
I2C (Inter-Integrated Circuit)

- Philips tarafından TV'ler için geliştirilmiştir.
- Master-slave konfigürasyonu ile birden fazla aygıtla kısa mesafeli senkron haberleşme yapabilmektedir.
- Genellikle bilgisayar aygıtlarının haberleşmesinde kullanılmazlar.
- Çoklu master yapısını da destekleyebilir.
- SDA ve SCL olmak üzere iki kablolu hatta sahiptir.
- Haberleşme hızı ~400Kbps'dir. Daha hızlı vers. vardır.
- Pull-up dirençleri en önemli parçasıdır.

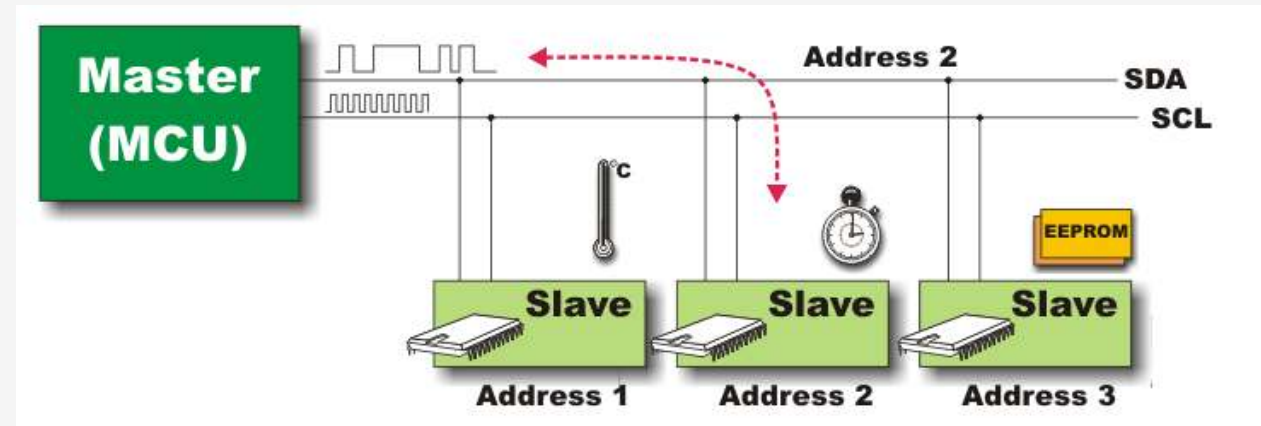


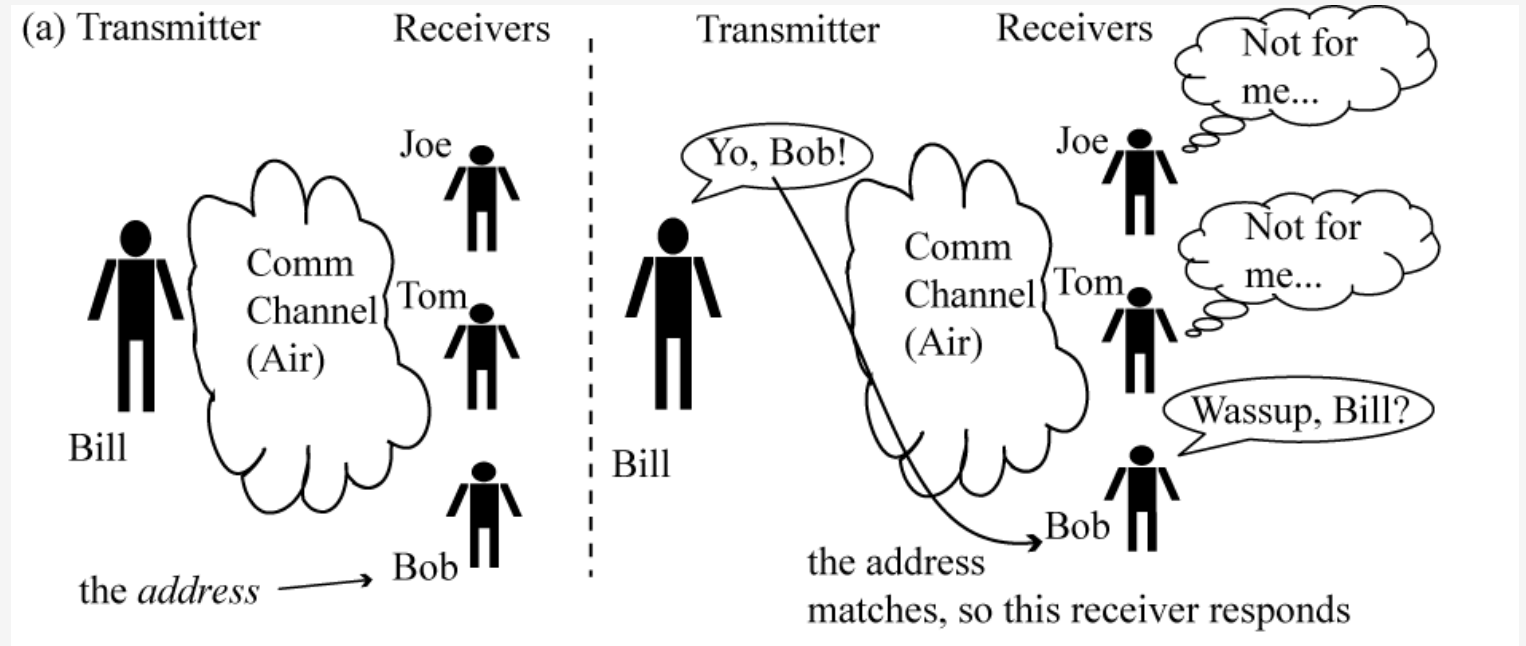
I2C (Inter-Integrated Circuit)

- 127 veya 1023 tane düğüm noktasına sahip olabilir.
- SDA (Serial Data Line) ve SCL (Serial Clock Line)
- Her bir bit, bir clockta iletilmektedir. ->



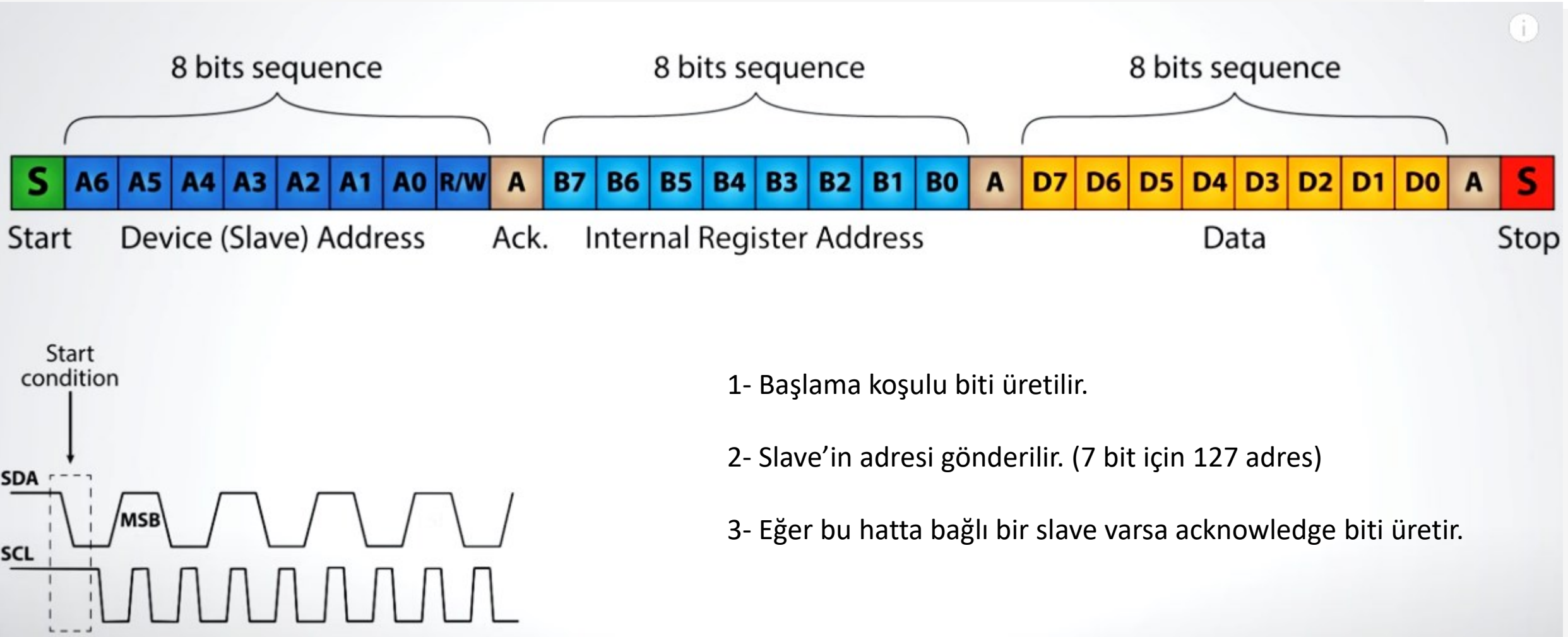
- SDA üzerindeki veri clock periyodu boyunca stabil olmalıdır.
- Tüm slave cihazlar aynı iletim hattını paylaşır ve ilk byte ı aynı zamanda alır.
- Fakat sadece bir tanesi bu adresle uyuşur.





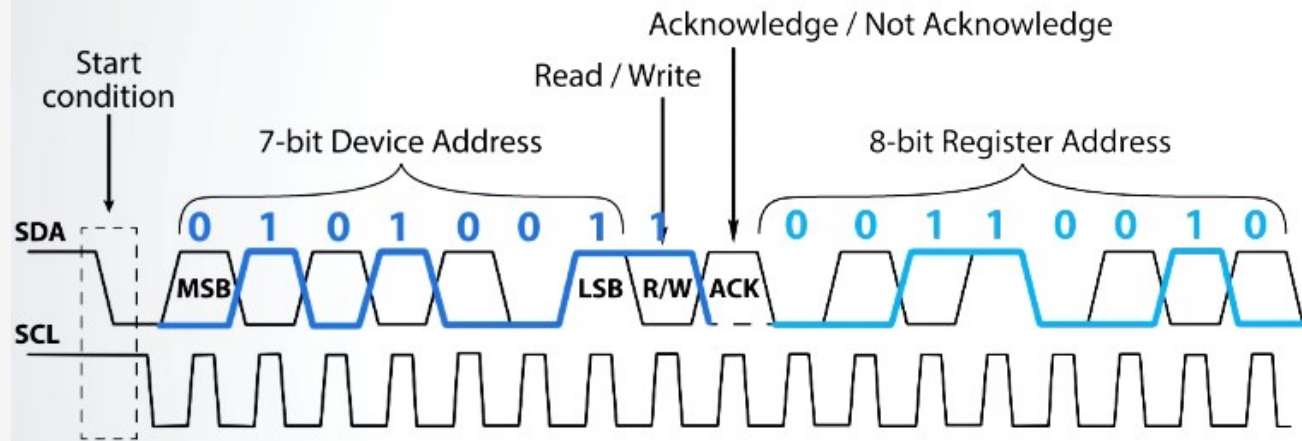
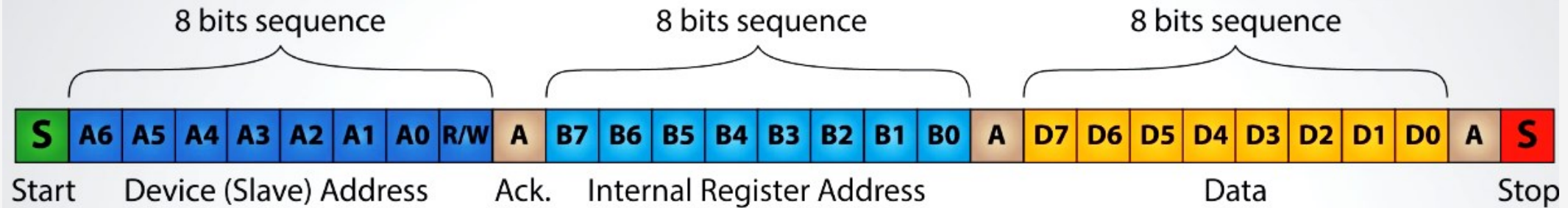
© Charles River
Media 2005

I2C Haberleşme



- 1- Başlama koşulu biti üretilir.
- 2- Slave'in adresi gönderilir. (7 bit için 127 adres)
- 3- Eğer bu hatta bağlı bir slave varsa acknowledge biti üretir.

I2C Haberleşme



4- Slave'de ulaşmak istediğimiz register adresi gönderilir.

5- Slave bu istek geçerli ise cevap verir.

For example: ADXL345 Accelerometer

Device Address: _____ 0x53 or 0101 0011 (read mode, 8th bit high)

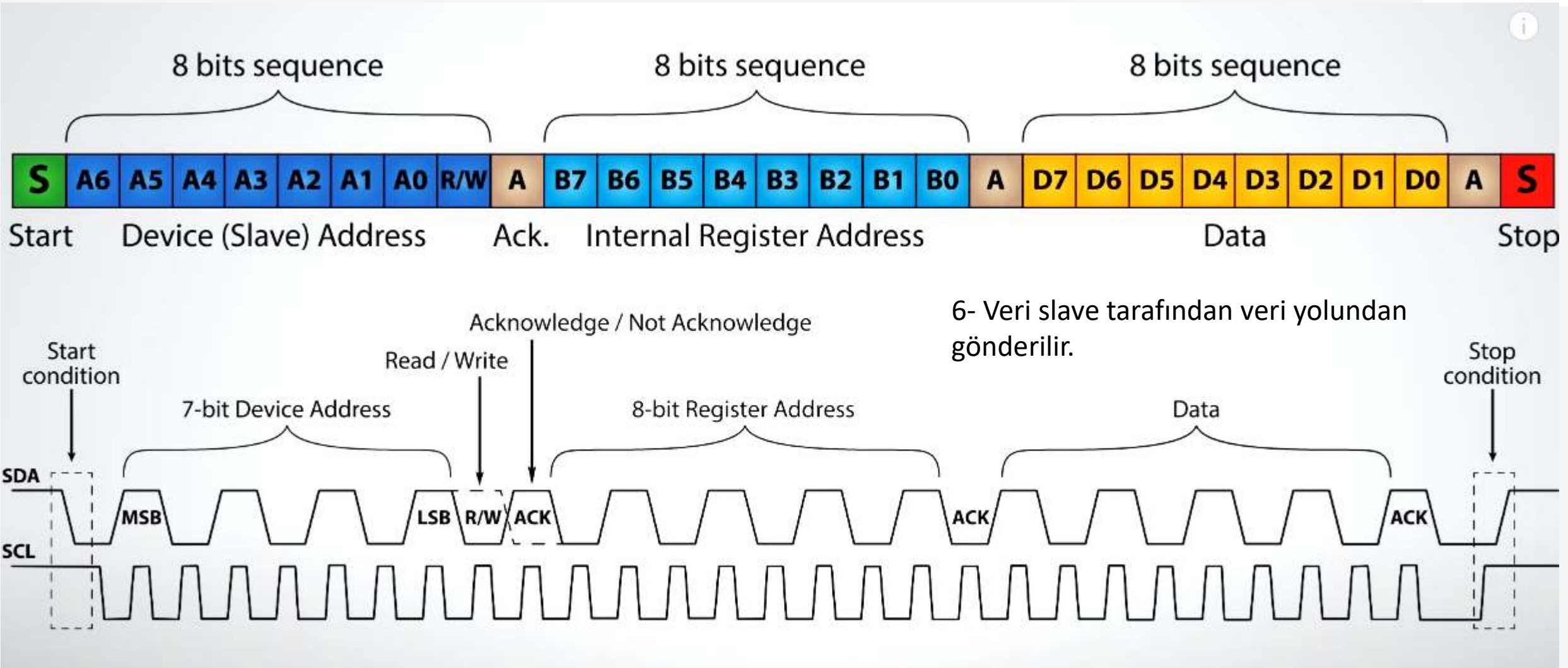
Internal Register Address

for the X Axis: _____ 0x32 or 0011 0010

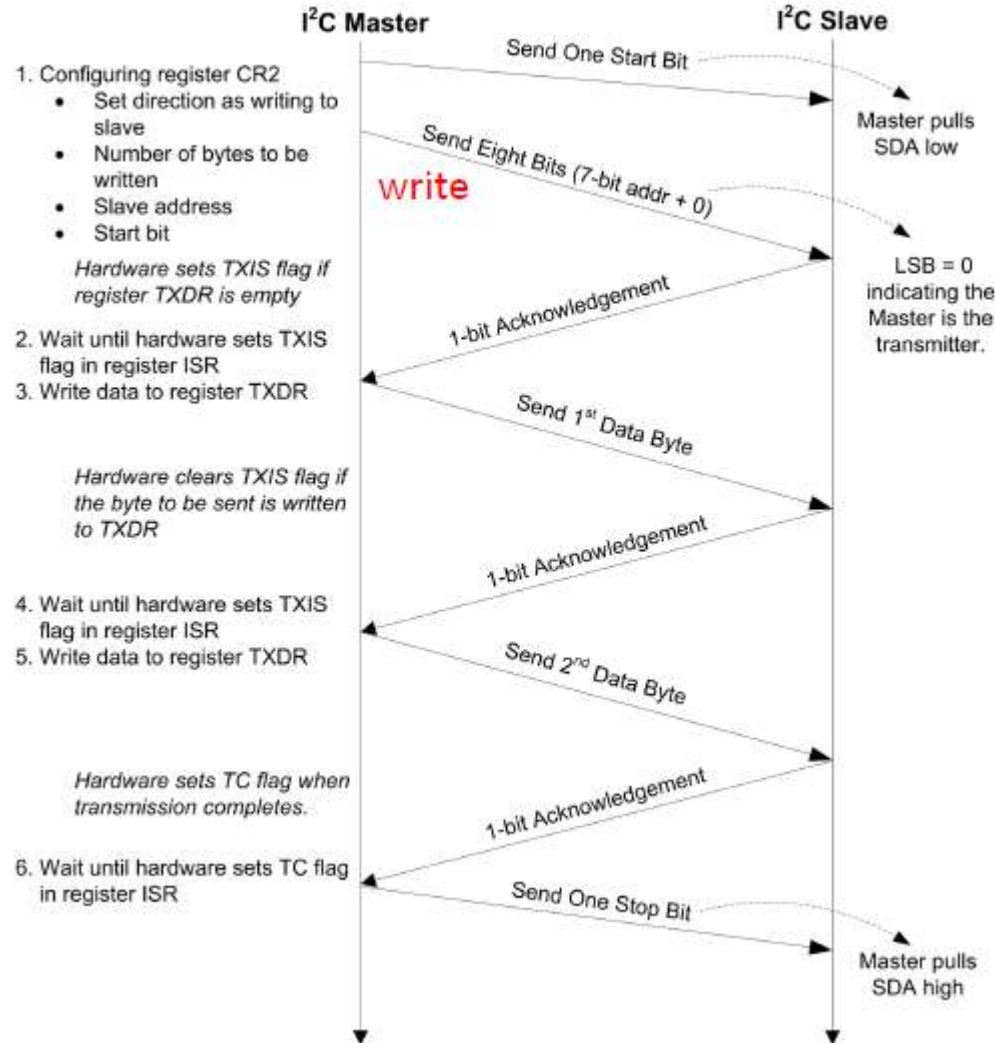


SUBSCRIBE
NOW!

I2C Haberleşme



Embedded C code



```
i2c_start();    // I2C İletişimi başlat
i2c_write(0xa0); // Slave konumundaki cihaz adresi
i2c_write(5);   // Komut gönderiliyor
i2c_write(12);  // Data (veri) gönderiliyor
i2c_stop();     // I2C iletişimi durduruluyor
```

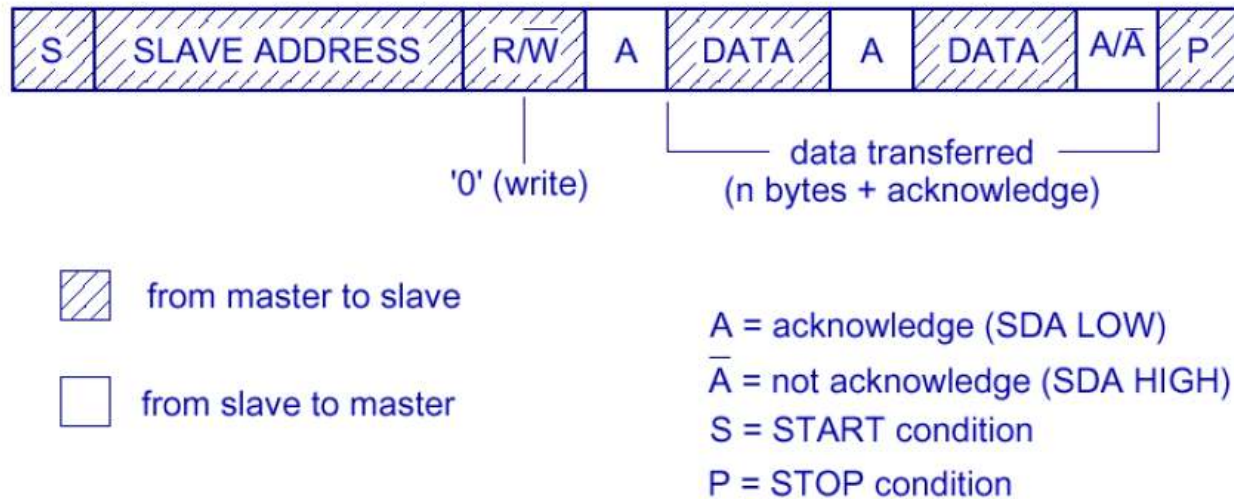
Operation	Description	Example
I2C WRITE	Send a single byte	<code>i2c_write(outbyte);</code>
I2C READ	Read a received byte	<code>inbyte=i2c_read();</code>
I2C STOP	Issue a stop command in master mode	<code>i2c_stop();</code>
I2C POLL	Check to see if byte received	<code>sbit=i2c_poll();</code>

Read or Write Commands



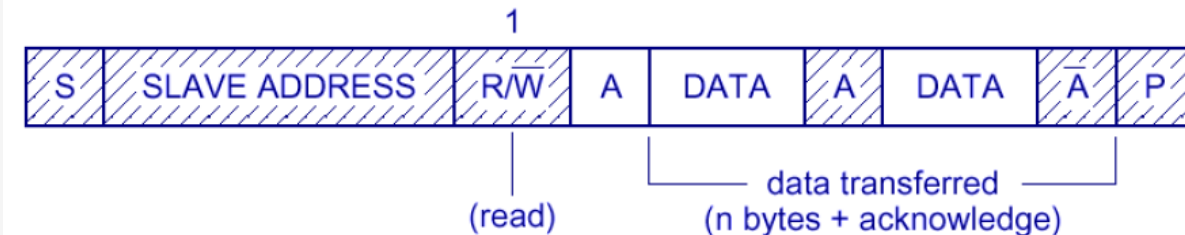
Master to Slave Data Transfer

- In this configuration, the transmission direction never changes



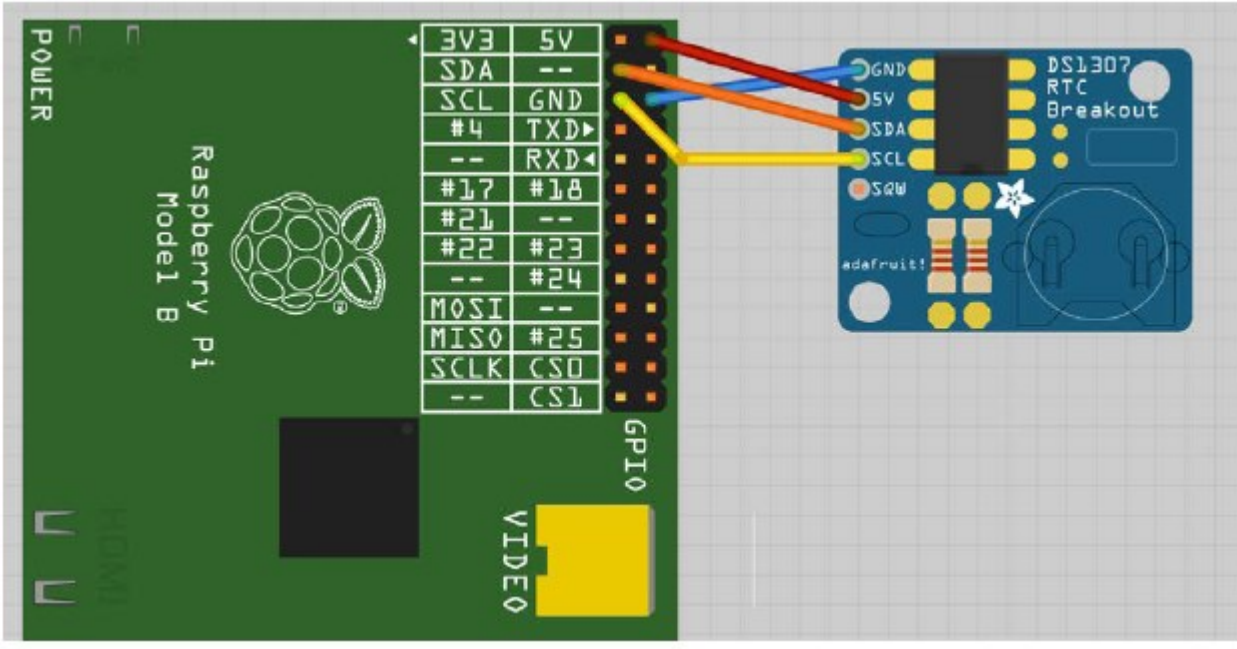
Slave to Master Data Transfer

- Master initiates the data transfer by generating the START condition followed by the start byte (with read/write bit set to 1, i.e. read mode)
- After the first ack from the slave, the direction of data changes and the master becomes receiver and slave transmitter.
- The STOP condition is still generated by the master (master sends not-ACK before generating the STOP)



Example: Reading Information from RTC

- Raspberry Pi sürekli internete bağlı değilse zamanı tutması gerekmektedir.



```
$ sudo i2cdetect -y 1|
    0 1 2 3 4 5 6 7 8 9 a b c d e f
00:          -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- 68 -- -- -- --
70: -- -- -- -- -- -- -- -- -- --
```

The 68 in the table indicates that the RTC module is connected to the I2C bus at hex address 68.

Example: Reading Information from RTC



Kod satırı:

```
$ sudo modprobe rtc-ds1307          # Loads RTC kernel driver for DS1307
```

```
$ sudo bash
```

```
$ echo ds1307 0x68 > /sys/class/i2c-adapter/i2c-1/new_device  # Connect the RTC to System I2C 1 Bus
```

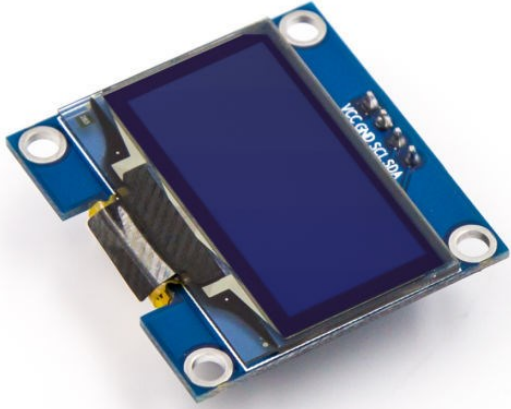
```
$ sudo hwclock -r  # Read the date from the RTC
```

```
Sat 01 Jan 2000 00:08:08 UTC -0.293998 seconds
```

```
$ sudo hwclock -w  # Write the date from system date
```

```
Sat 06 Jan 2021 10:08:08 UTC -0.293998 seconds
```

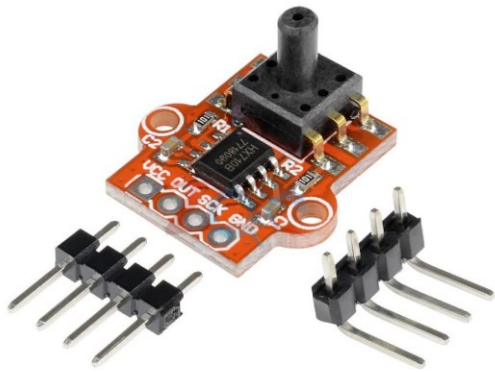

Example-2



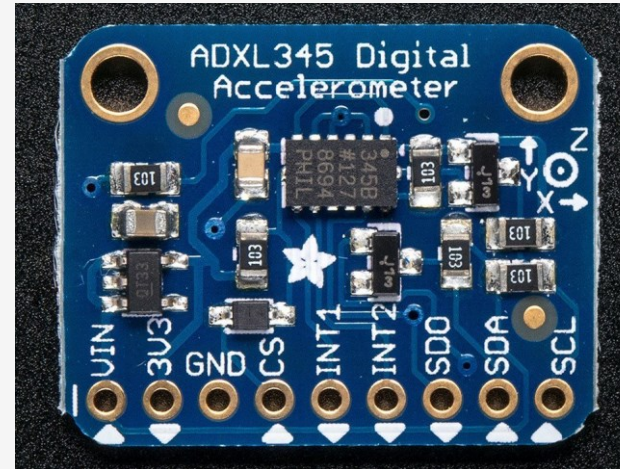
OLED ekran



NFC sensörü



Basınç sensörü

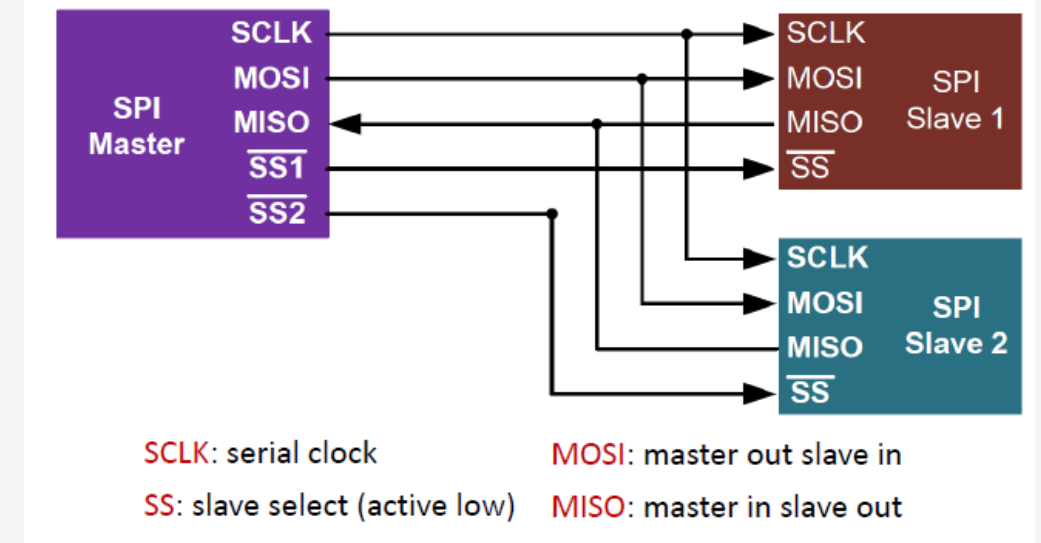


İvme ölçer

SPI (Serial Peripheral Interface)



- Senkron, full-duplex ve seri haberleşme
- En az 4 I/O gereklidir.
- Tek master fakat birden fazla slave ağıta uyumludur.
- Herhangi bir akış kontrol (ACK) biti yok.
- Birkaç Mbps hızında çalışabilirler.
- ADC dönüştürücüleri gibi data streamler için uygundur.
- Eğer tek bir slave varsa SS pini lojik sıfıra sabitlenebilir.



SPI (Serial Peripheral Interface)



I2C'de her veri alışverişinde slave in adresi gerekiyordu.

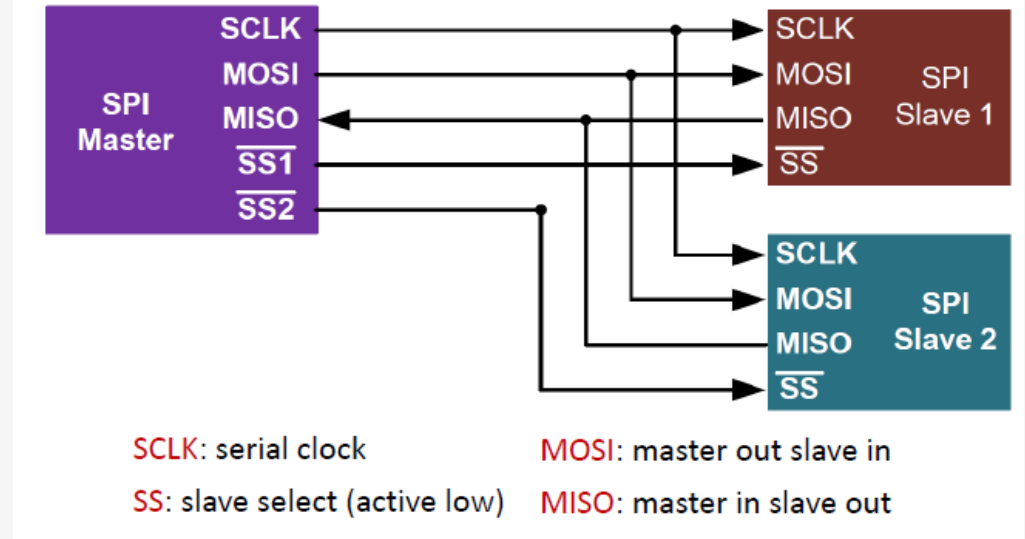
Burada, SS tek bitte bu olayı çözüyor.

Pekçok sensör tarafından kullanılmaktadır.

Dezavantaj: ACK biti yok, çoklu aygıt durumunda problem

Akış:

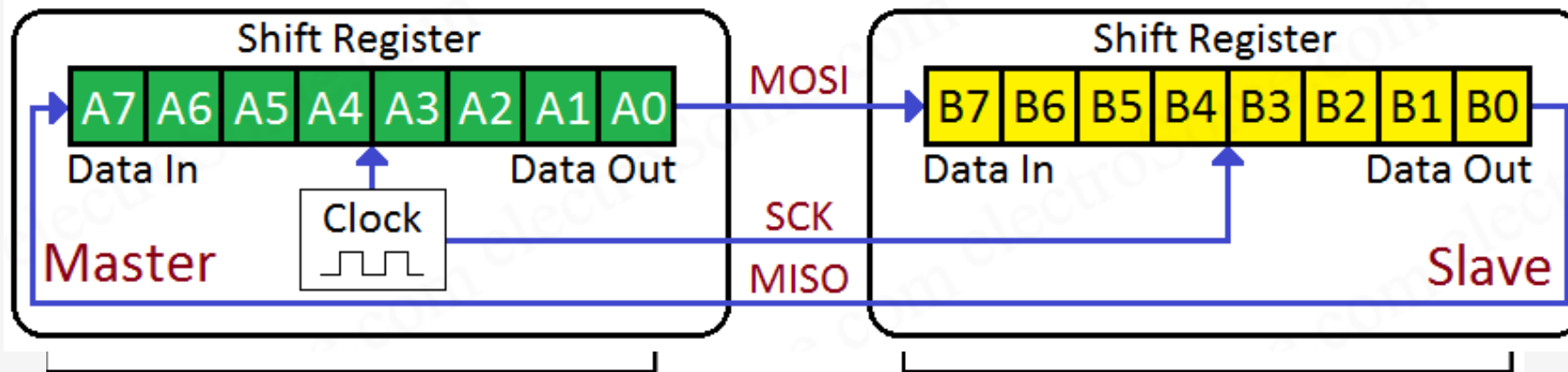
- 1- SCLK konfigüre edilir. Clock frekansı iletişim hızını belirler.
- 2- SS ile ilgili aygıtı bağlı pin lojik sıfıra çekilir.
- 3- MISO ve MOSI eşzamanlı transferi yapar.
- 4- İşlem bittikten sonra SS tekrar lojik 1 yapılır.



SPI (Serial Peripheral Interface)



Data Exchange

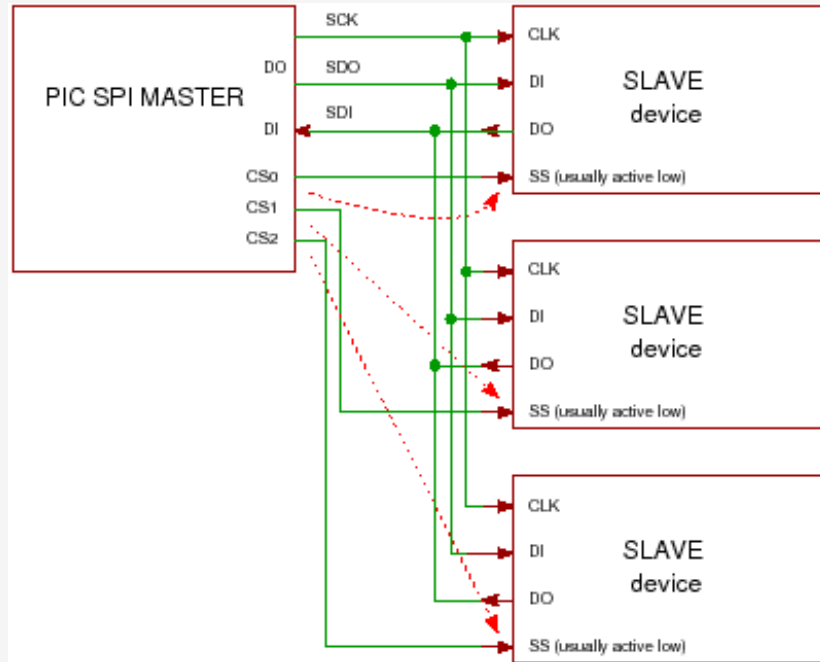


- MOSI – Carries data out of Master to Slave
- MISO – Carries data from Slave to Master

- Both signals happen for every transmission

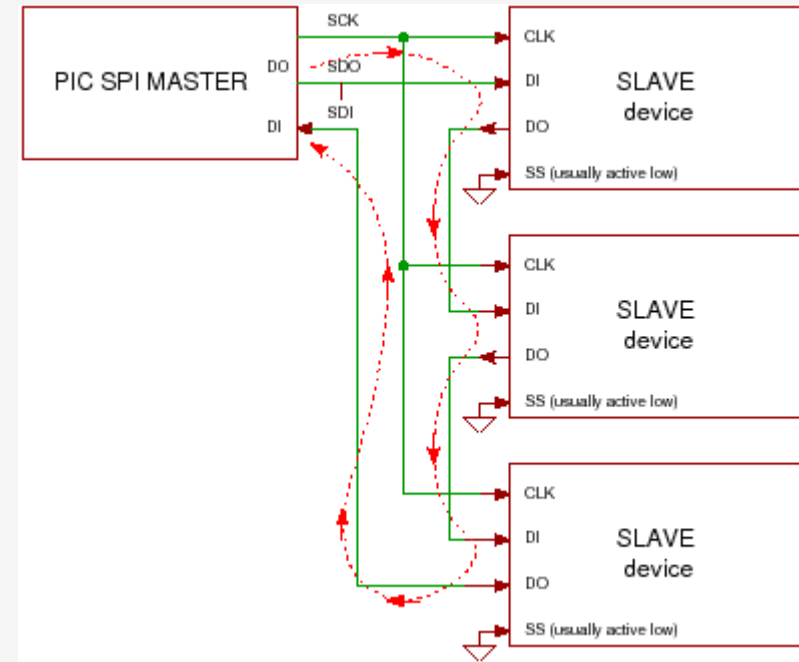
- Master has to provide clock to slave
- **Synchronous exchange**: for each clock pulse, a bit is shifted out and another bit is shifted in at the same time. This process stops when all bits are swapped.
- Only **master can start the data transfer**

Diagram



Master and multiple independent slaves

http://upload.wikimedia.org/wikipedia/commons/thumb/f/fc/SPI_three_slaves.svg/350px-SPI_three_slaves.svg.png



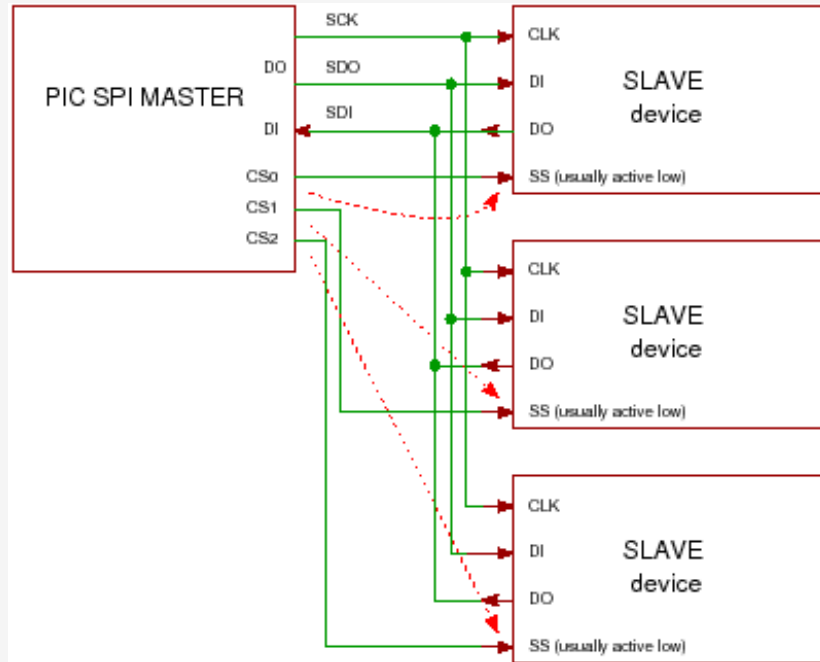
Some wires have been renamed

Master and multiple daisy-chained slaves

http://www.maxim-ic.com/appnotes.cfm/an_pk/3947

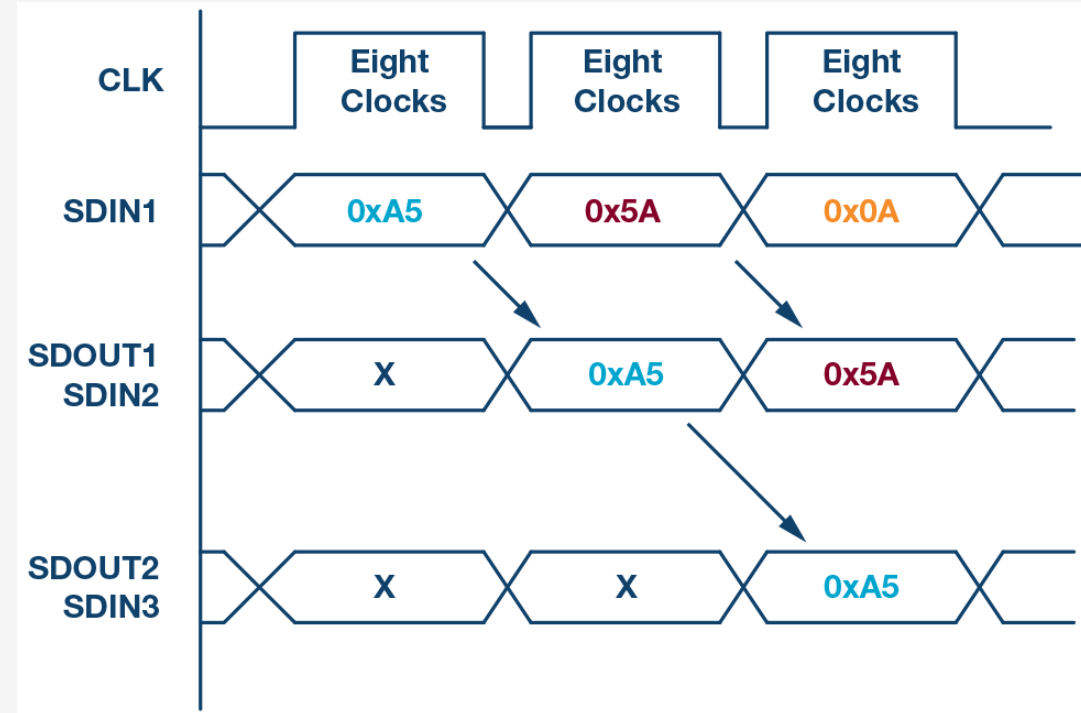
Master tarafından gönderilen veri slave cihazlara kaydırılarak verilir. Bunun çalışabilmesi için clock kullanımı bütün cihazlarda aynı olmalı ve slave cihazlar tarafından alınan bitlerin sayısı doğru olmalıdır. Yazılım maliyeti artmaktadır.

Diagram



Master and multiple independent slaves

http://upload.wikimedia.org/wikipedia/commons/thumb/f/fc/SPI_three_slaves.svg/350px-SPI_three_slaves.svg.png



Some wires have been renamed

Master and multiple daisy-chained slaves

http://www.maxim-ic.com/appnotes.cfm/an_pk/3947

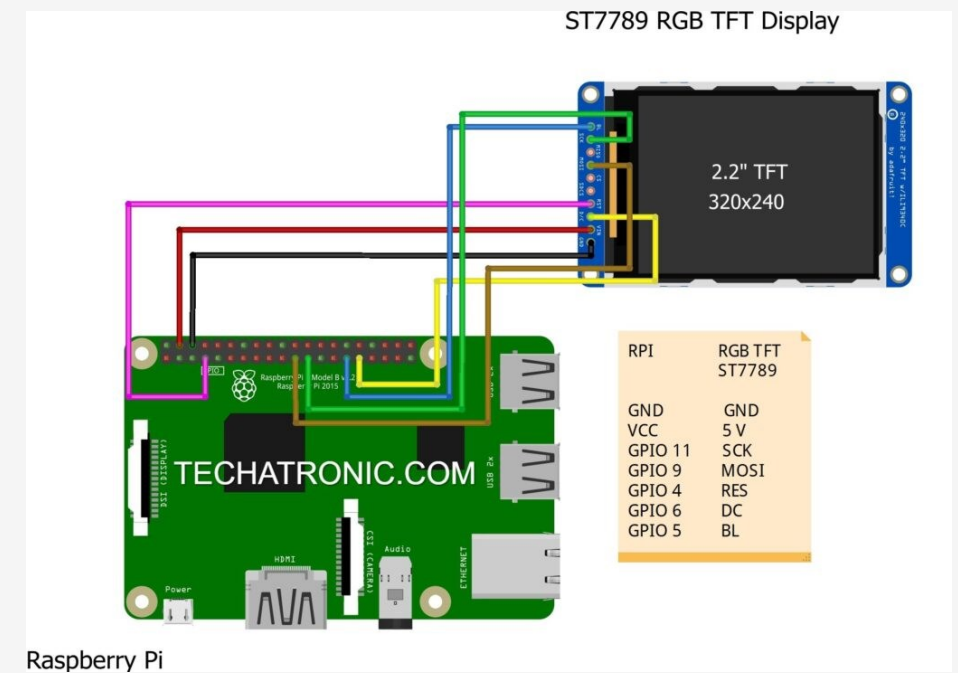
Master tarafından gönderilen veri slave cihazlara kaydırılarak verilir. Bunun çalışabilmesi için clock kullanımı bütün cihazlarda aynı olmalı ve slave cihazlar tarafından alınan bitlerin sayısı doğru olmalıdır. Yazılım maliyeti artmaktadır.

Example: TFT Display

ST7789 is the driver IC of the TFT display. These types of displays come in various sizes. 128×240, 240×240, 240×320 are the resolution in which it is available.

```
sudo pip3 install st7789
sudo apt-get install python3-rpi.gpio
sudo apt-get install python3-spidev
sudo apt-get install python3-pil
sudo apt-get install python3-numpy
```

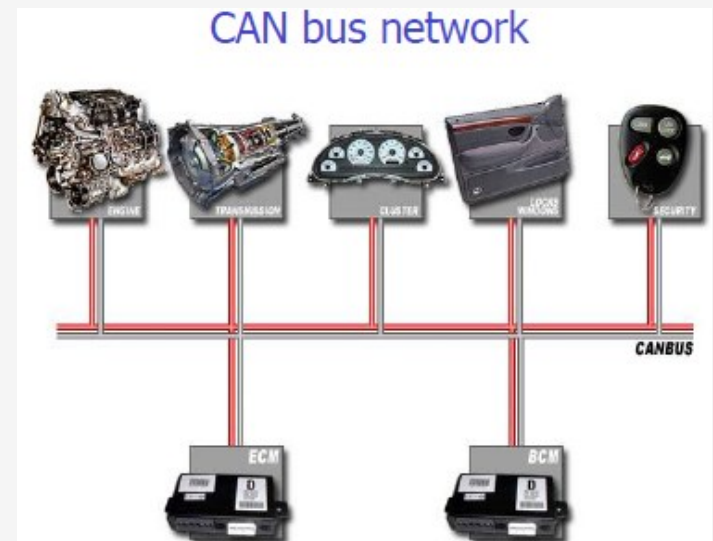
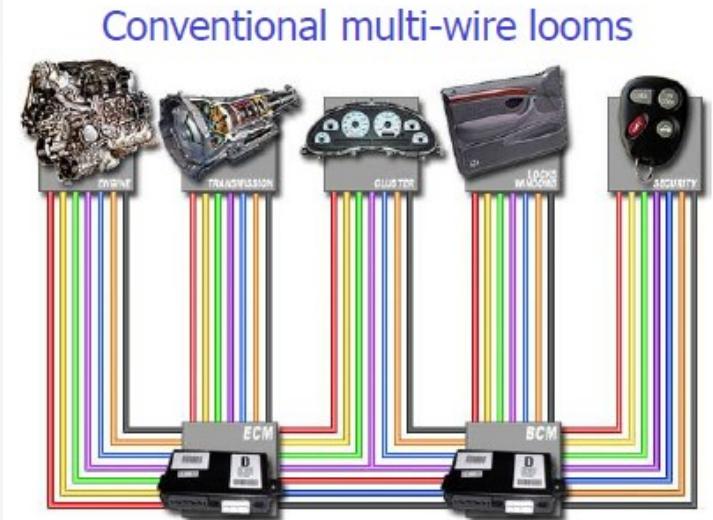
```
import ST7789
from PIL import Image
from time import sleep
display=ST7789.ST7789(port=0,cs=0,rst=6,dc=5,backlight=4,spi_speed_hz=160000000)
display._spi.mode=3
display.reset()
display._init()
image=Image.new('RGB',(240,240),(255,0,0)) #('RGB',(240,240),(r,g,b))
display.display(image)
sleep(2)
image=Image.open("image5.jpg")
image=image.resize((240,240),resample=Image.LANCZOS)
display.display(image)
```



CAN (Controller Area Network)

- Controller Area Network (CAN) is a fast serial bus that is designed to provide
 - an efficient,
 - reliable and
 - very economical link between sensors and actuators
- Introduced by Bosch in 1980s
- CAN uses a twisted pair cable (dual-wire) to communicate at speeds up to 1Mbit/s (max) with up to 40 devices
- It originally developed to simplify the wiring in automobiles
- CAN (and other fieldbus standards) is now used in machine and factory automation products as well

Credit by Zambreno



CAN (Controller Area Network)



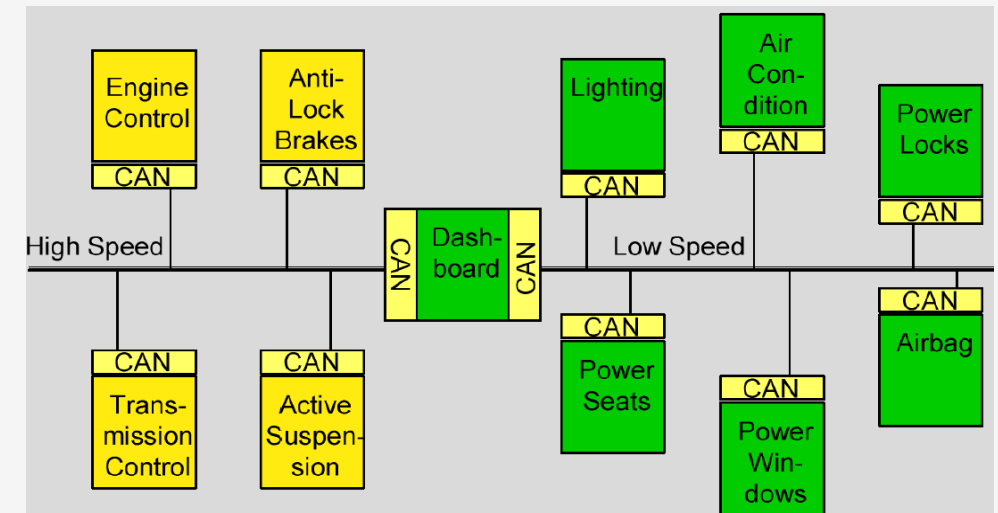
CAN uses a twisted pair cable to communicate at speeds up to **1Mbit/s** with up to **40 devices**. (~40m)

CAN Field buses are now used in machine and factory automation products as well.

Any node can access the bus when the bus is quiet

Automatic error detection, signaling and retries

Data packets are 8 bytes long



CAN Message



SOF → Start of the packet

CAN-ID → Message ID → Priority & Functional Address (example RPM – Real Speed)

RTR = 1 ise mesaj karşıdan isteniyor, RTR = 0 ise mesaj veri içeriyor.

Control → Informs the length of the data in bytes

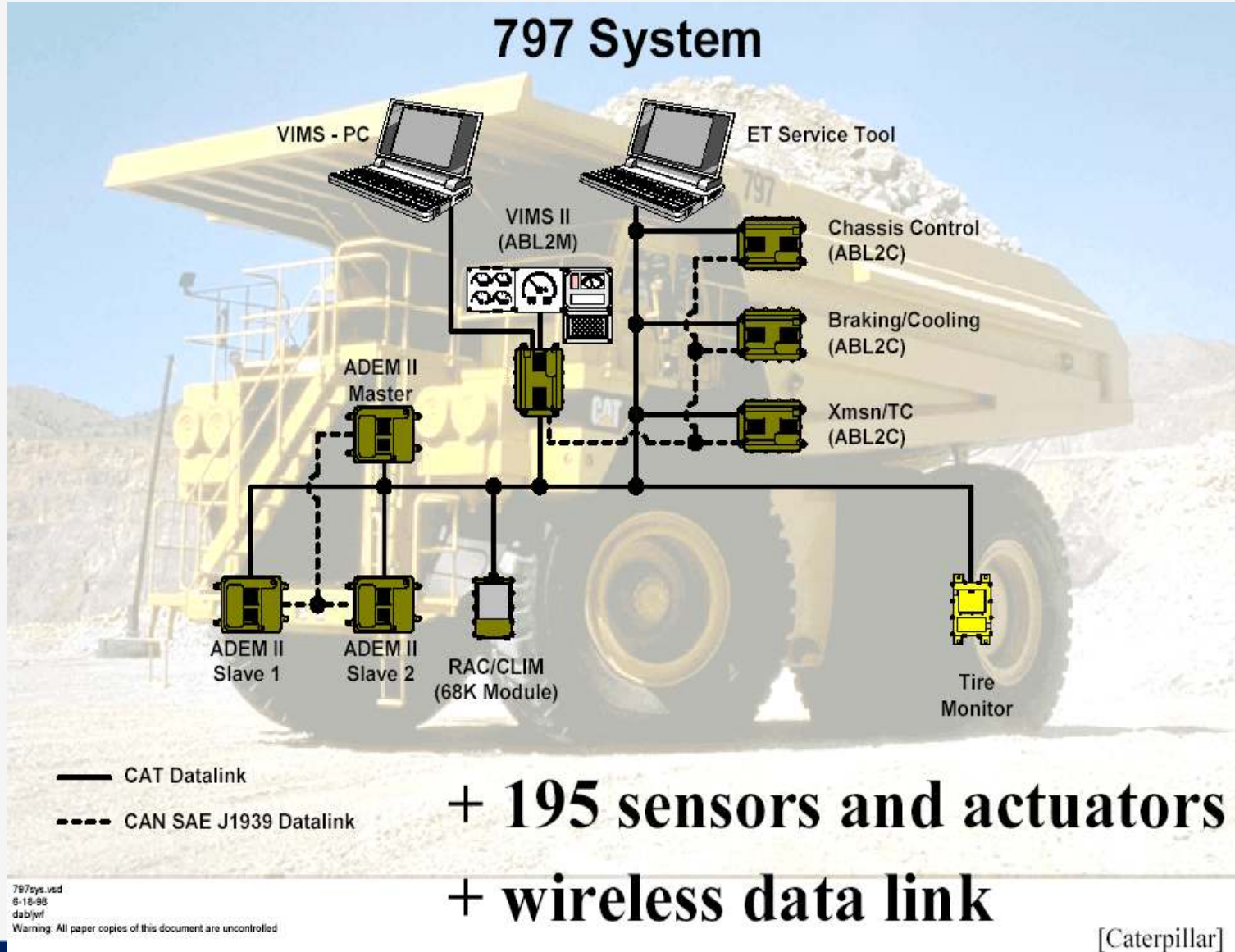
CRC → Double check (Alan kişi SOF-CRC ye kadar bir hesap yapar yaptığı hesap CRC ile karşılaştırır.)

ACK → Mesajı alan var mı yok mu?

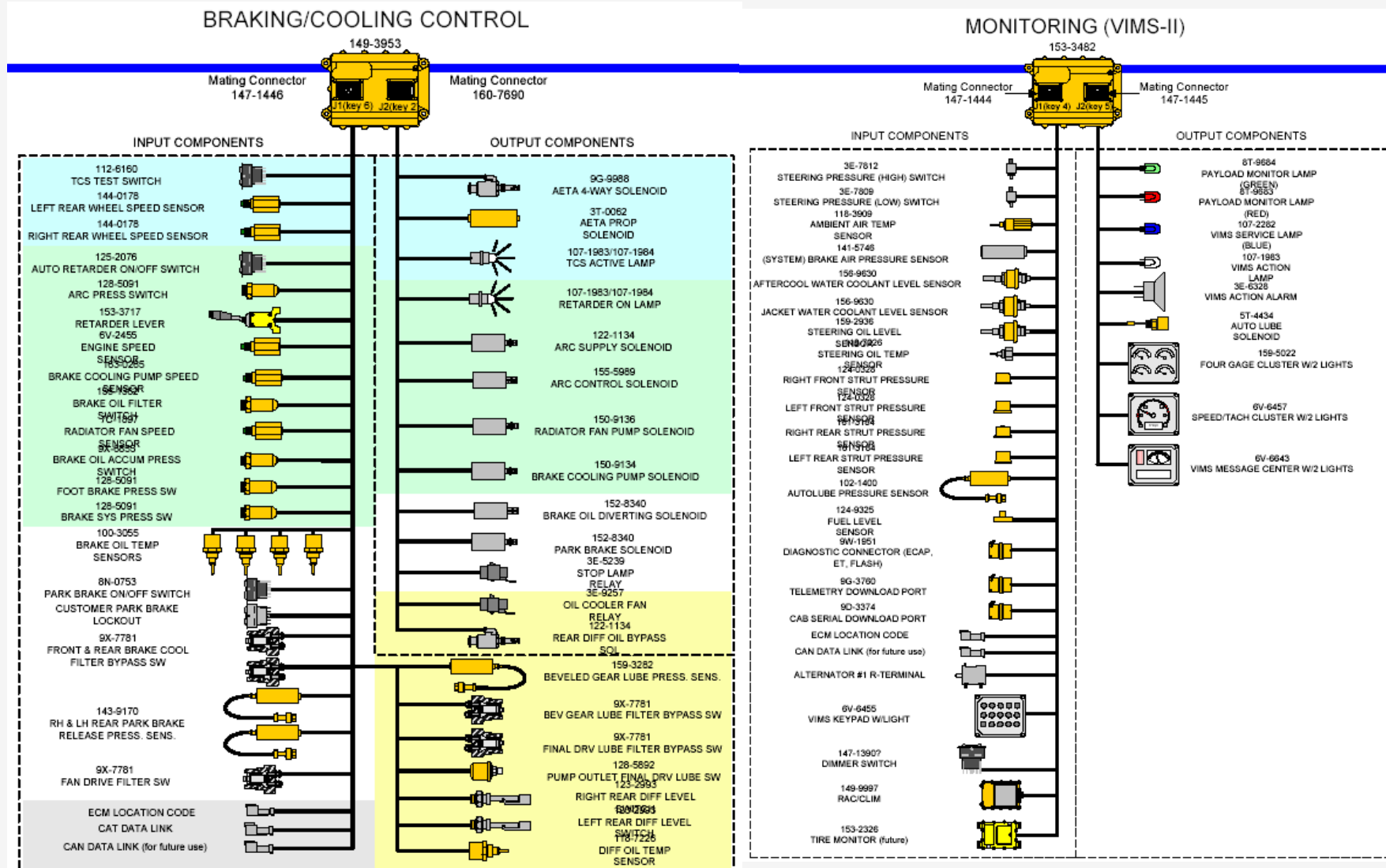


Protocol	Expandable	Multi-Master	Duplex	Robustness	Speed	Flow Control	Protocol Overhead
UART (2-wire)	No	No	Full	(Parity bit)	~100Kbps	No	Start, Stop, (parity)
I2C	Large	Yes	Half	ACK/NACK	S:100Kbps F:400Kbps Hs:3.4Mbps	Yes	Start, 7-bit addr, R/W, ACK/NACK
SPI	Extra wire per slave	No	Full	None	8Mbps+	No	None
CAN	Large	Yes	Half	16-bit CRC ACK/NACK Differential signaling	1Mbps	No (for single frame)	Start, 11-bit ID, 6-bit control, 16-bit CRC, 2-bit ACK, 7-bit EOF

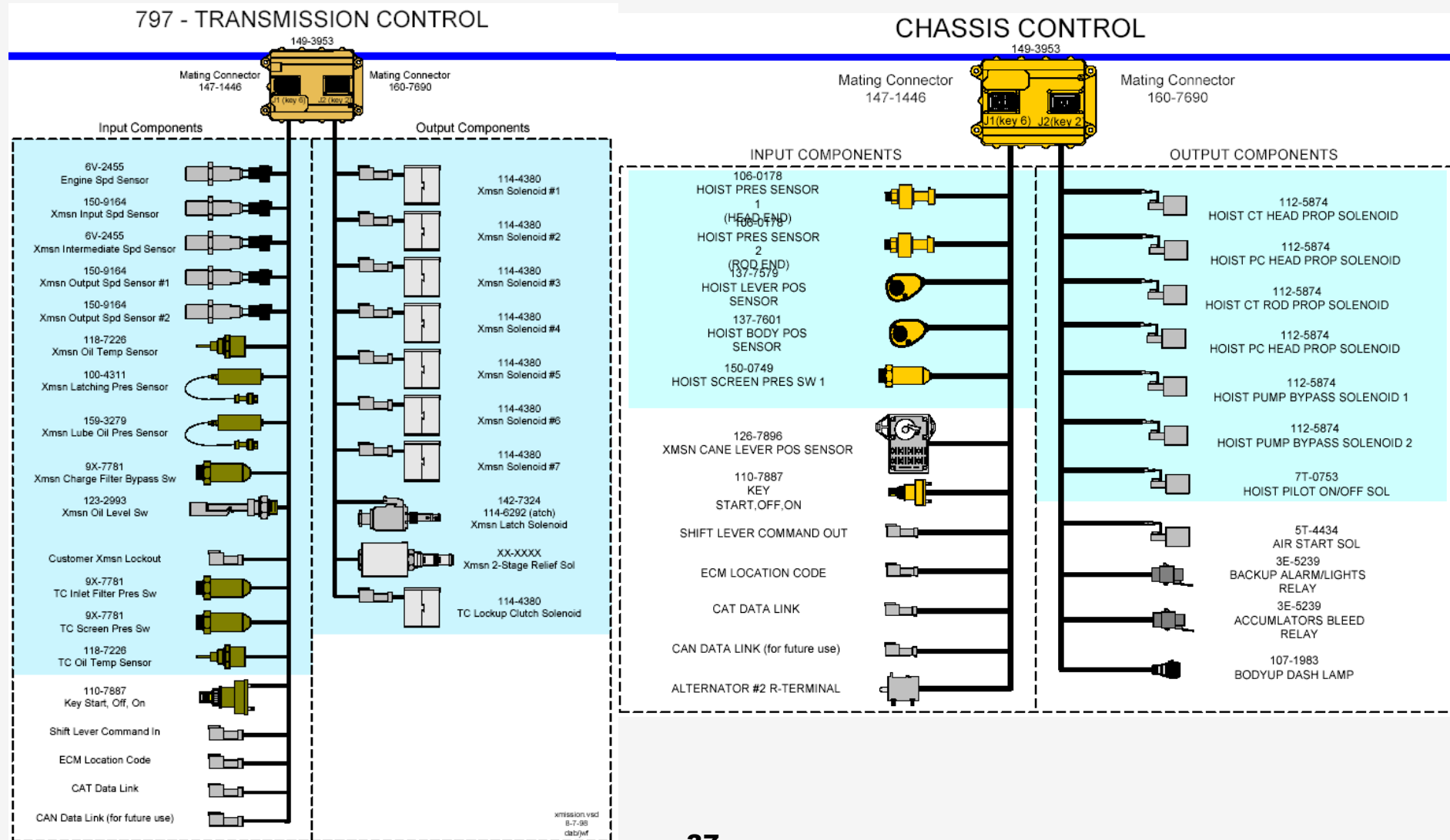
CAN (SAE J1939) Example: Caterpillar 797



Caterpillar example



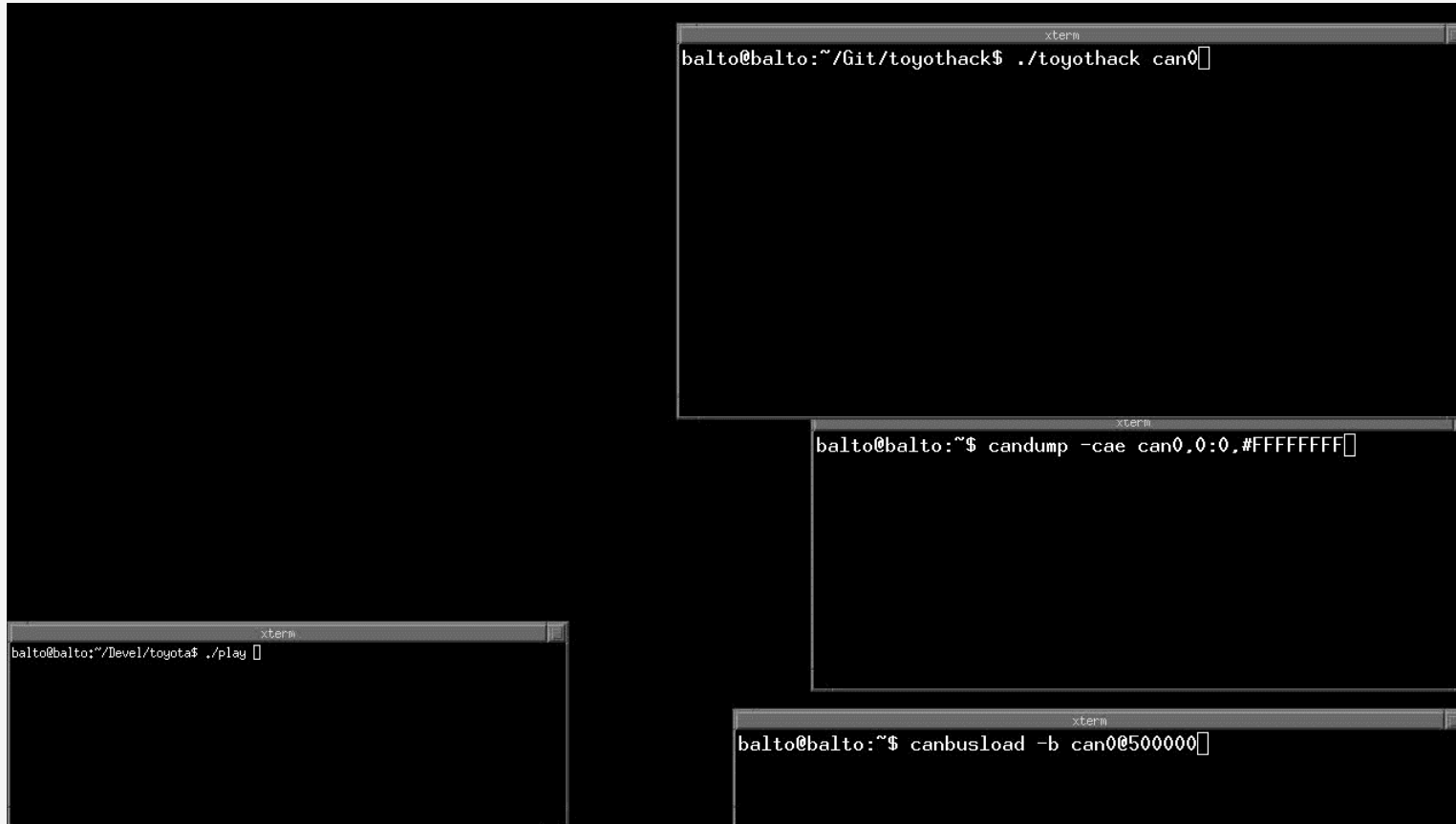
Caterpillar example



“SAE Standard Workload” (53 messages) V/C = Vehicle Controller

Signal Number	Signal Description	Size /bits	J /ms	T /ms	Periodic /Sporadic	D /ms	From	To
1	Traction Battery Voltage	8	0.6	100.0	P	100.0	Battery	V/C
2	Traction Battery Current	8	0.7	100.0	P	100.0	Battery	V/C
3	Traction Battery Temp, Average	8	1.0	1000.0	P	1000.0	Battery	V/C
4	Auxiliary Battery Voltage	8	0.8	100.0	P	100.0	Battery	V/C
5	Traction Battery Temp, Max.	8	1.1	1000.0	P	1000.0	Battery	V/C
6	Auxiliary Battery Current	8	0.9	100.0	P	100.0	Battery	V/C
7	Accelerator Position	8	0.1	5.0	P	5.0	Driver	V/C
8	Brake Pressure, Master Cylinder	8	0.1	5.0	P	5.0	Brakes	V/C
9	Brake Pressure, Line	8	0.2	5.0	P	5.0	Brakes	V/C
10	Transaxle Lubrication Pressure	8	0.2	100.0	P	100.0	Trans	V/C
11	Transaction Clutch Line Pressure	8	0.1	5.0	P	5.0	Trans	V/C
12	Vehicle Speed	8	0.4	100.0	P	100.0	Brakes	V/C
13	Traction Battery Ground Fault	1	1.2	1000.0	P	1000.0	Battery	V/C
14	Hi&Lo Contactor Open/Close	4	0.1	50.0	S	5.0	Battery	V/C
15	Key Switch Run	1	0.2	50.0	S	20.0	Driver	V/C
16	Key Switch Start	1	0.3	50.0	S	20.0	Driver	V/C
17	Accelerator Switch	2	0.4	50.0	S	20.0	Driver	V/C
18	Brake Switch	1	0.3	20.0	S	20.0	Brakes	V/C
19	Emergency Brake	1	0.5	50.0	S	20.0	Driver	V/C
20	Shift Lever (PRNDL)	3	0.6	50.0	S	20.0	Driver	V/C
21	Motor/Trans Over Temperature	2	0.3	1000.0	P	1000.0	Trans	V/C
22	Speed Control	3	0.7	50.0	S	20.0	Driver	V/C
23	12V Power Ack Vehicle Control	1	0.2	50.0	S	20.0	Battery	V/C
24	12V Power Ack Inverter	1	0.3	50.0	S	20.0	Battery	V/C
25	12V Power Ack I/M Contr.	1	0.4	50.0	S	20.0	Battery	V/C
26	Brake Mode (Parallel/Split)	1	0.8	50.0	S	20.0	Driver	V/C

Hacking CAN bus?

Three terminal windows are displayed on a black background. The top window shows the command './toyothonack can0'. The middle window shows 'candump -cae can0,0:0,#FFFFFFF'. The bottom window shows 'canbusload -b can0@500000'. Each window has a title bar with the word 'xterm'.

<https://fabiobaltieri.com/2013/07/23/hacking-into-a-vehicle-can-bus-toyothack-and-socketcan/>